

Maze Solver

Documentação Técnica do Projeto

Linguagem	Python 3.14
Biblioteca	pygame-ce 2.5.7
Algoritmo	A* (A-estrela)
Geração	Recursive Backtracking (DFS)
Controles	WASD para mover · R para reiniciar

1. Visão Geral do Projeto

O **Maze Solver** é um jogo interativo em que o jogador navega por um labirinto gerado aleatoriamente usando as teclas WASD. Ao primeiro movimento, o algoritmo **A*** calcula e exibe o caminho ótimo até a saída, servindo como dica visual. O projeto utiliza **pygame-ce** para renderização e conta com animação suave de movimento (interpolação de pixels a 60 fps).

Estrutura de arquivos

Arquivo	Responsabilidade
main.py	Loop principal, entrada do jogador, animacao, tela de vitoria
config.py	Constantes globais: dimensoes, cores e velocidade
cell.py	Classe Cell — representa cada celula do labirinto
maze_generator.py	Criacao do grid e geracao do labirinto (DFS)
astar.py	Implementacao do algoritmo A*
renderer.py	Funcoes de desenho: labirinto, caminho, marcadores S/F

2. config.py — Constantes Globais

Centraliza todos os valores de configuração do projeto. Alterar aqui reflete automaticamente em todos os outros módulos.

```
LARGURA = 600          # largura/altura da janela em pixels
LINHAS    = 20           # numero de celulas por linha/coluna
TAM        = LARGURA // LINHAS # tamanho de cada celula (30 px)

# Tema escuro
FUNDO      = (18, 18, 30)  # cor de fundo do labirinto
COR_CELULA = (28, 28, 46)  # interior de cada celula
COR_PAREDE = (80,120,200)  # cor das paredes (azul-arroxeadado)

# Jogador
COR_JOGADOR      = ( 60, 200, 255)
COR_JOGADOR_BORDA = ( 20,  80, 160)
COR_BRILHO        = (180, 230, 255)

# Caminho A*
COR_CAMINHO      = (100, 255, 160)
COR_CAMINHO_HALO = ( 50, 140,  90)

# Pontos inicio / fim
COR_INICIO = ( 60, 220,  80)
COR_FIM    = (255,  70,  70)
```

TAM é derivado de **LARGURA / LINHAS**, portanto mudar qualquer um dos dois reescala todo o labirinto automaticamente. Para um labirinto menor ou maior, basta alterar **LINHAS**.

3. cell.py — A Célula do Labirinto

Define a classe **Cell**, que representa cada quadrado do grid. Cada célula guarda sua posição e quais de suas quatro paredes ainda estão ativas (True = parede existe).

```
class Cell:
    def __init__(self, i, j):
        self.i = i          # coluna (eixo X)
        self.j = j          # linha  (eixo Y)
        self.paredes = {
            "top":    True,
            "bottom": True,
            "left":   True,
            "right":  True,
        }
        self.visitado = False # usado na geracao do labirinto

    def __lt__(self, outro):
        return False          # necessario para PriorityQueue do A*
```

O método `__lt__` retorna sempre **False** para evitar erro de comparação quando dois nós têm o mesmo `f_score` na fila de prioridade do A*.

4. maze_generator.py — Geração do Labirinto

Responsável por criar o grid de células e gerar o labirinto usando o algoritmo **Recursive Backtracking** (DFS iterativo com pilha explícita). Esse método garante que todo labirinto gerado tem solução única.

4.1 criar_grid()

```
def criar_grid():  
    # Retorna uma matriz [LINHAS][LINHAS] de objetos Cell  
    return [[Cell(i, j) for j in range(LINHAS)] for i in range(LINHAS)]
```

4.2 gerar_labirinto(grid)

Implementa o algoritmo DFS para 'escavar' passagens entre células adjacentes, removendo paredes compartilhadas:

```
def gerar_labirinto(grid):  
    stack = []  
    atual = grid[0][0]  
    atual.visitado = True  
  
    while True:  
        # Coleta vizinhos nao visitados  
        vizinhos_nv = [...]  
  
        if vizinhos_nv:  
            proximo = random.choice(vizinhos_nv)  
            stack.append(atual)  
  
            # Remove a parede entre 'atual' e 'proximo'  
            dx = atual.i - proximo.i  
            dy = atual.j - proximo.j  
            if dx == 1: atual.paredes["left"] = False; proximo.paredes["right"] = False  
            elif dx == -1: atual.paredes["right"] = False; proximo.paredes["left"] = False  
            elif dy == 1: atual.paredes["top"] = False; proximo.paredes["bottom"] = False  
            elif dy == -1: atual.paredes["bottom"] = False; proximo.paredes["top"] = False  
  
            atual = proximo  
            atual.visitado = True  
        elif stack:  
            atual = stack.pop() # retrocede (backtrack)  
        else:  
            break  
  
    # Abre entrada (topo) e saída (baixo)  
    grid[0][0].paredes["top"] = False  
    grid[LINHAS-1][LINHAS-1].paredes["bottom"] = False
```

A abertura do topo e da base cria a entrada e saída visíveis no labirinto.

5. astar.py — Algoritmo A*

Implementa o algoritmo de busca de caminho **A*** (**A-estrela**), que encontra o caminho mais curto entre a célula inicial e a célula final respeitando as paredes do labirinto.

5.1 heuristica(a, b)

```
def heuristica(a, b):  
    # Distancia de Manhattan: |xi - xj| + |yi - yj|  
    return abs(a.i - b.i) + abs(a.j - b.j)
```

A distância de Manhattan é a heurística ideal para grades sem movimento diagonal.

5.2 vizinhos(cell, grid)

```
def vizinhos(cell, grid):  
    lista = []  
    i, j = cell.i, cell.j  
    # Verifica cada direcao: so adiciona se NAO ha parede entre as celulas  
    if j > 0 and not cell.paredes["top"]:  
        lista.append(grid[i][j-1])  
    if j < LINHAS-1 and not cell.paredes["bottom"]:  
        lista.append(grid[i][j+1])  
    if i > 0 and not cell.paredes["left"]:  
        lista.append(grid[i-1][j])  
    if i < LINHAS-1 and not cell.paredes["right"]:  
        lista.append(grid[i+1][j])  
    return lista
```

5.3 a_star(grid, inicio, fim)

Executa o A* e retorna a lista de células que formam o caminho ótimo:

```
def a_star(grid, inicio, fim):  
    fila = PriorityQueue()      # min-heap por f_score  
    fila.put((0, count, inicio))  
  
    came_from = {}              # rastreia o caminho  
    g_score = {cell: inf ...}   # custo real ate cada no  
    f_score = {cell: inf ...}   # g + heuristica  
  
    while not fila.empty():  
        atual = fila.get()[2]  
  
        if atual == fim:  
            # Reconstroi o caminho de tras para frente  
            caminho = []  
            while atual in came_from:  
                caminho.append(atual)  
                atual = came_from[atual]  
            caminho.append(inicio)  
            caminho.reverse()  
            return caminho  
  
        for vizinho in vizinhos(atual, grid):  
            temp_g = g_score[atual] + 1  
            if temp_g < g_score[vizinho]:
```

```
came_from[vizinho] = atual
g_score[vizinho] = temp_g
f_score[vizinho] = temp_g + heuristica(vizinho, fim)
fila.put((f_score[vizinho], count, vizinho))
```

6. renderer.py — Renderização

Concentra todas as funções de desenho na tela, separando a lógica visual do loop principal.

6.1 desenhar(tela, grid, caminho)

Limpa a tela, pinta o fundo e o interior das células, desenha os pontos do caminho A* (halo + ponto central) e por último renderiza as paredes por cima:

```
def desenhar(tela, grid, caminho=None):
    tela.fill(FUNDO)

    # Fundo de cada célula
    for linha in grid:
        for cell in linha:
            pygame.draw.rect(tela, COR_CELULA, (x+1, y+1, TAM-2, TAM-2))

    # Caminho sugerido (bolinhas com halo)
    if caminho:
        for cell in caminho:
            pygame.draw.circle(tela, COR_CAMINHO_HALO, (cx, cy), TAM//5)
            pygame.draw.circle(tela, COR_CAMINHO, (cx, cy), TAM//8)

    # Paredes sobre tudo
    for linha in grid:
        for cell in linha:
            if cell.paredes["top"]: pygame.draw.line(...)
            if cell.paredes["bottom"]: pygame.draw.line(...)
            if cell.paredes["left"]: pygame.draw.line(...)
            if cell.paredes["right"]: pygame.draw.line(...)
```

6.2 desenhar_pontos(tela, inicio, fim)

Desenha os marcadores de início (S) e fim (F) como círculos coloridos com letra:

```
def _bandeira(tela, cx, cy, cor):
    pygame.draw.circle(tela, cor, (cx, cy), TAM//3)
    pygame.draw.circle(tela, BRANCO, (cx, cy), TAM//3, 2)
    fonte = pygame.font.SysFont("consolas", TAM//3, bold=True)
    letra = "S" if cor == COR_INICIO else "F"
    surf = fonte.render(letra, True, BRANCO)
    tela.blit(surf, ...)
```


7. main.py — Loop Principal

Orquestra todos os módulos: inicializa o pygame, gerencia o estado do jogo, processa entradas e exibe os elementos visuais a 60 fps.

7.1 mover_jogador(grid, jogador, tecla)

```
movimentos = {
    pygame.K_w: ("top", 0, -1),
    pygame.K_s: ("bottom", 0, 1),
    pygame.K_a: ("left", -1, 0),
    pygame.K_d: ("right", 1, 0),
}
parede, di, dj = movimentos[tecla]
if jogador.paredes[parede]:
    return jogador # movimento bloqueado por parede
return grid[i+di][j+dj] # retorna a celula destino
```

7.2 Animação Suave de Movimento

Em vez de teletransportar o jogador, a posição pixel é interpolada a cada frame em direção ao centro da célula alvo:

```
VELOCIDADE_ANIM = TAM * 0.18 # pixels por frame

# A cada frame:
alvo_x, alvo_y = cell_center(jogador)
dx, dy = alvo_x - px, alvo_y - py
dist = math.hypot(dx, dy)
if dist > VELOCIDADE_ANIM:
    px += dx / dist * VELOCIDADE_ANIM # anda na direcao certa
    py += dy / dist * VELOCIDADE_ANIM
else:
    px, py = alvo_x, alvo_y # chegou
```

7.3 desenhar_jogador(tela, px, py, direcao, tick)

Desenha o personagem com efeitos visuais:

```
bob = math.sin(tick * 0.12) * 1.5 # balanço vertical suave

# Sombra elíptica abaixo do personagem
pygame.draw.ellipse(sombra, (0,0,0,60), ...)

# Corpo (círculo azul com borda e brilho)
pygame.draw.circle(tela, COR_JOGADOR, (cx, cy), raio)
pygame.draw.circle(tela, COR_JOGADOR_BORDA, (cx, cy), raio, 2)
pygame.draw.circle(tela, COR_BRILHO, (cx-raio//3, cy-raio//3), raio//4)

# Olhos virados para a direção do movimento
ox, oy = offsets[direcao] # deslocamento dos olhos
pygame.draw.circle(tela, BRANCO, olho_esq, olho_r)
pygame.draw.circle(tela, PRETO, olho_esq, pupila_r)
```

7.4 Tela de Vitória

```
def desenhar_vitoria(tela, tick):
```

```

# Overlay escuro pulsante
alpha = int(180 + 30 * math.sin(tick * 0.08))
overlay = pygame.Surface((LARGURA, LARGURA), pygame.SRCALPHA)
overlay.fill((0, 0, 0, alpha))
tela.blit(overlay, (0, 0))

# Texto com efeito de sombra/glow
for dx, dy in [(-1,-1),(1,-1),(-1,1),(1,1)]:
    tela.blit(sombra, (x+dx, y+dy)) # sombra colorida
tela.blit(titulo, ...)             # texto principal

```

7.5 Fluxo do Loop Principal

```

while rodando:
    clock.tick(60)          # limita a 60 fps
    tick += 1

    # 1. Interpola posicao pixel do jogador
    # 2. Desenha labirinto + caminho + marcadores
    # 3. Desenha jogador na posicao interpolada
    # 4. Se venceu, sobrepoem tela de vitoria
    # 5. Processa eventos (WASD, R, fechar janela)
    pygame.display.update()

```

8. Fluxo Geral de Execução

Etapa	O que acontece
1. Inicializacao	pygame.init() + janela 600x600 + clock 60fps
2. Gerar labirinto	criar_grid() => Cell[20][20]; gerar_labirinto() (DFS)
3. Aguarda input	Jogador pressiona WASD pela primeira vez
4. Calcular caminho	a_star(grid, inicio, fim) -> lista de Cell
5. Mover jogador	Verifica parede; atualiza celula alvo; interpola pixels
6. Atualizar caminho	Remove celulas ja visitadas da dica visual
7. Verificar vitoria	jogador == fim => ativa tela de vitoria pulsante
8. Reiniciar (R)	Gera novo grid + labirinto; reseta todos os estados

O caminho do A* é calculado apenas **uma vez** por partida (no primeiro movimento) e as células já visitadas pelo jogador são removidas da dica em tempo real.